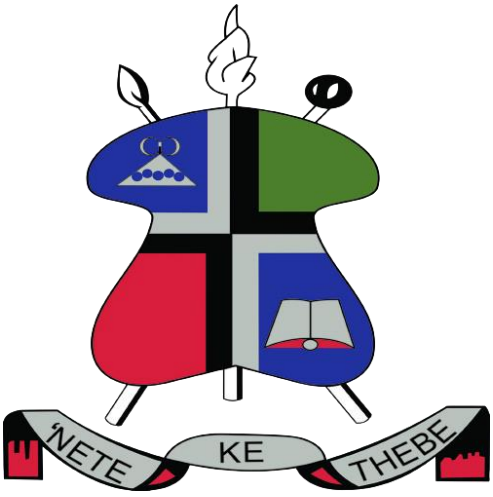



# National University of Lesotho

Faculty of Science and Technology

Department of Mathematics and Computer Science

Verification and Validation - Phase 6

# Learning Management System


| | |
|---|---|
| Submitted by | MAFATANE. T |
| Student number | 202321187 |
| Course | CS4433 - Software Engineering |
| Submission date | 30/10/2025 |

# 1. Verification and Validation

This phase evaluates whether the Learning Management System (LMS) has been implemented correctly and whether it satisfies the requirements established during requirements engineering and system design. The verification and validation approach follows the V-Model and includes a test strategy, automated unit and integration testing, system and acceptance testing, code coverage analysis, and defect management.

Evidence currently available from the project includes 68 passing automated unit and integration tests, 78% business-logic line coverage, and manual testing of administrator login and access to the administration interface. The report distinguishes executed tests from scenarios for which execution evidence is still required.

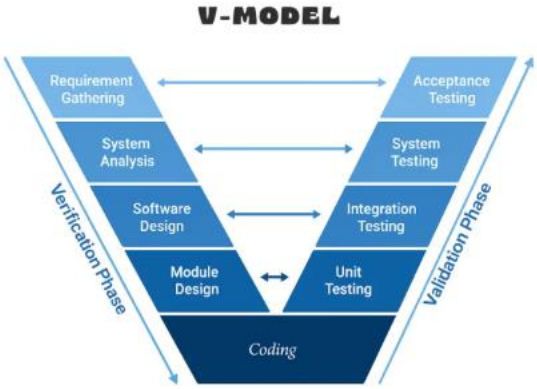
# 1.1 Test Strategy

## 1.1.1 Testing objectives

- Verify that individual business-logic components operate according to their specifications.
- Verify that integrated components communicate correctly with the SQLite database.
- Verify complete LMS workflows across the student, lecturer, HoD and administrator roles.
- Validate that the system satisfies the functional requirements and use cases.
- Identify defects, correct them and confirm the corrections through retesting.

## 1.1.2 V-Model testing approach

The project adopts the V-Model because it links development artefacts to corresponding testing activities. Requirements analysis is validated through acceptance testing; system design through system testing; architectural design through integration testing; and module design through unit testing.




## 1.1.3 Test levels, artefacts and entry/exit criteria

| Test level | Artefact verified | Entry criteria | Exit criteria |
|---|---|---|---|
| Unit testing | Individual classes and business-logic components, including AuthService, CourseService and LatePenaltyStrategy. | Component implementation and test requirements are available. | Required unit tests pass; failures are corrected or documented. |
| Integration testing | Interactions between Database, services, authentication, submission, marking and moderation components. | Individual components have passed their unit tests. | Integrated workflows execute correctly; database state and audit records are consistent. |
| System testing | The complete LMS application, including role-specific dashboards and management workflows. | Integrated components and a deployable application are available. | System test cases pass, including relevant exception flows; critical defects are resolved. |
| Acceptance testing | Functional requirements and use cases specified in the SRS. | System testing is sufficiently complete and acceptance scenarios are prepared. | Acceptance criteria are evaluated and the system is assessed against user requirements. |

### 1.1.4 Test techniques

#### A. Equivalence partitioning

Equivalence partitioning divides input values into groups expected to produce similar system behaviour. A representative value is selected from each group.

| Feature | Valid equivalence class | Invalid equivalence class |
|---|---|---|
| Login | Correct username and password | Incorrect username or password; missing credentials |
| Course creation | Unique, non-empty course code | Duplicate or empty course code |
| Assignment deadline | Valid deadline according to the assignment rules | Missing or invalid deadline |
| Assignment submission | Eligible, enrolled student submits an assignment | Unenrolled student or invalid submission |
| Mark entry | Mark between zero and maximum mark, inclusive | Negative mark or mark above maximum |
| User role | STUDENT, LECTURER, HOD or ADMIN | Unsupported role |

#### B. Boundary value analysis

Boundary value analysis tests values at and around input limits. It is particularly useful for assignment deadlines and late penalties. The following cases reflect the late-penalty rule stated for this project; confirm that the implementation and approved requirements use the same day-counting convention before final submission.

| ID | Input condition | Expected result |
|---|---|---|
| BVA-01 | Submission before deadline | No late penalty. |
| BVA-02 | Submission exactly at deadline | No late penalty. |
| BVA-03 | Submission one day late | 10% of maximum mark deducted. |
| BVA-04 | Submission three days late | 30% of maximum mark deducted. |
| BVA-05 | Submission more than three days late | Zero mark, according to the stated policy. |
| BVA-06 | Valid extension has been granted | Extended deadline is applied. |
| BVA-07 | Submission after extended deadline | Penalty is evaluated against the extended deadline. |

For a submission up to three days late, the penalty amount is calculated as:

$$\textbf{Penalty} = \min(0.10 \times \textbf{late days}, 1.00) \times \textbf{maximum mark}$$

## 1.2 Automated Unit and Integration Testing

### 1.2.1 Testing framework and environment

GoogleTest was selected as the automated testing framework because it supports structured test cases, assertions and test fixtures. CMake is used to build the application and test executables, while CTest discovers and executes the registered tests.

| Component | Technology |
|---|---|
| Programming language | C++17 |
| GUI framework | Qt 6 |
| Database | SQLite |
| Unit testing framework | GoogleTest |
| Build system | CMake |
| Test execution | CTest |
| Compiler | GCC |
| Coverage tooling | GCC coverage instrumentation and gcovr |

## 1.2.2 Unit testing

Unit tests verify individual components independently. The recorded unit-test suite contains 66 tests across authentication, course validation, late-penalty calculation and database queries.

| Component | Test focus | Test count |
|---|---|---|
| AuthService | Authentication and credential validation | 7 |
| CourseService | Course validation and business rules | 24 |
| LatePenaltyStrategy | Deadline calculations and late penalties | 9 |
| Database queries | Database retrieval and query behaviour | 26 |
| Total | | 66 |

Source files used:

- tests/unit/AuthServiceTests.cpp
- tests/unit/CourseServiceTests.cpp
- tests/unit/LatePenaltyTests.cpp
- tests/unit/DatabaseQueryTests.cpp

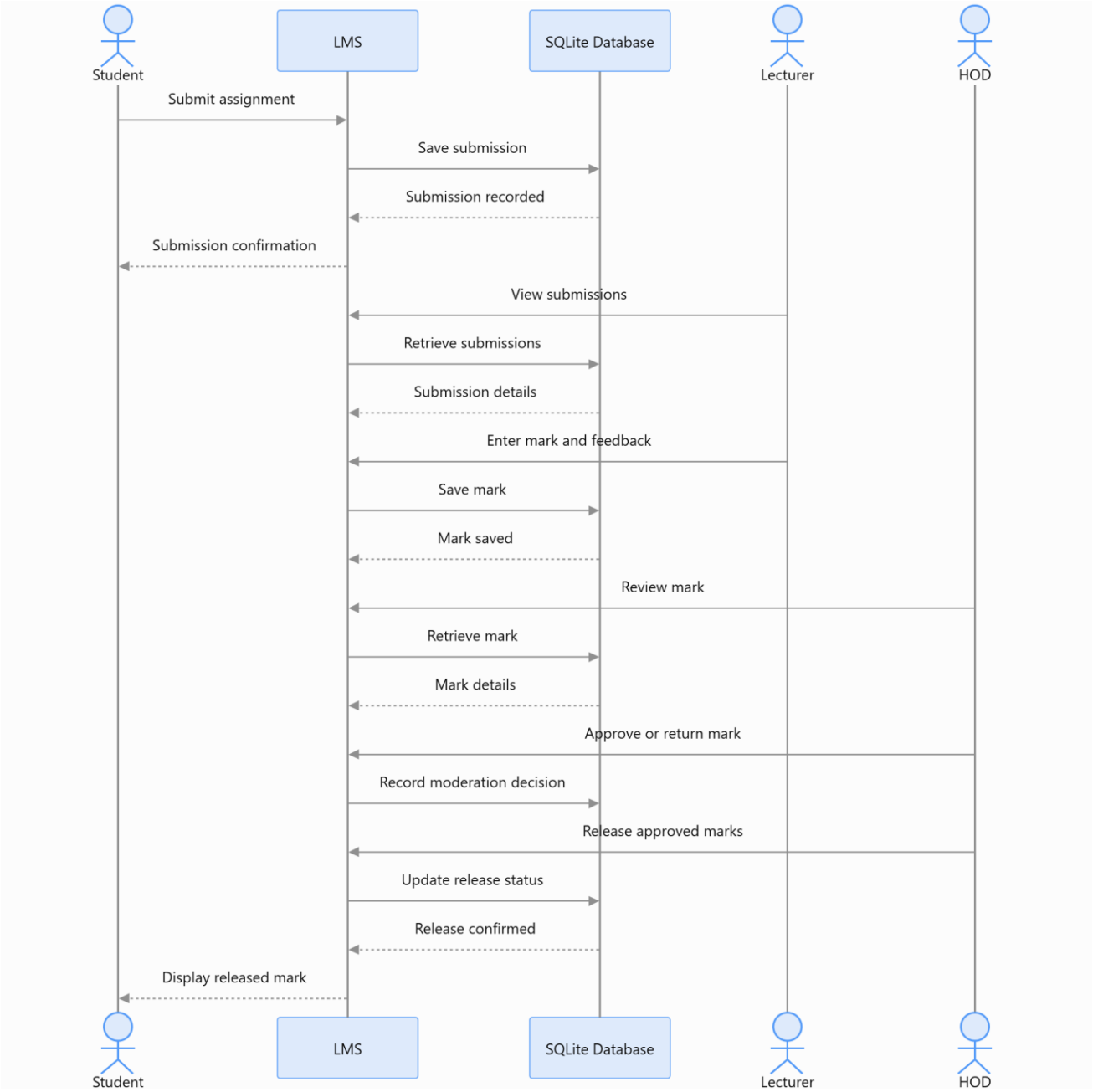
## 1.2.3 Integration testing

Integration tests verify that multiple LMS components work together to complete important workflows. Two integration tests are registered.

| Test ID | Integration test | Purpose | Recorded result |
|---|---|---|---|
| INT-01 | LMS Workflow Integration | Exercises assessment activities from setup and assignment submission through marking, moderation and release. | PASS |
| INT-02 | LMS Audit Trail Integration | Checks audit event generation, filtering and access restrictions. | PASS |

The assessment workflow integration test covers:

- Creation of users, departments and courses, lecturer assignment and student enrolment.
- Assignment creation, extension management, submission and resubmission.
- Submission attempt limits and superseded-submission handling.
- Lecturer marking, HoD moderation, approval and release of marks.
- Student retrieval of released marks and feedback.
- Late-submission penalty handling.




## 1.2.4 Automated test execution results

The automated suite was executed using the following command:

```
ctest --test-dir build --output-on-failure
```

| Test category | Passed | Failed | Total |
|---|---|---|---|
| Authentication | 7 | 0 | 7 |
| Course service | 24 | 0 | 24 |
| Late penalty | 9 | 0 | 9 |
| Database queries | 26 | 0 | 26 |
| Integration workflows | 2 | 0 | 2 |
| Total | 68 | 0 | 68 |

All 68 registered automated unit and integration tests passed in the recorded execution. Attach a terminal screenshot showing the CTest summary (100% tests passed, 0 tests failed out of 68) and retain the full test output as a project artefact.

## 1.2.5 Code coverage analysis

Code coverage was measured using GCC coverage instrumentation and gcovr. The objective was to measure the proportion of business-logic source lines executed by the automated tests.

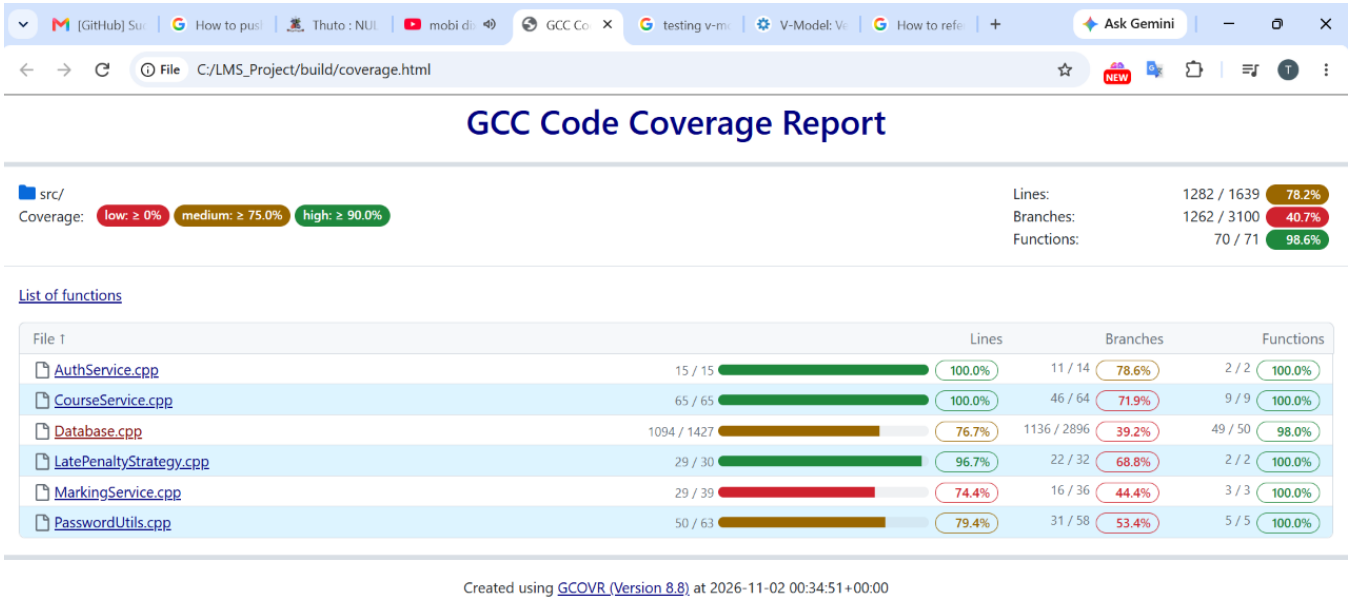
| Source file | Lines | Executed | Coverage |
|---|---|---|---|
| AuthService.cpp | 15 | 15 | 100% |
| CourseService.cpp | 65 | 65 | 100% |
| Database.cpp | 1427 | 1094 | 76% |
| LatePenaltyStrategy.cpp | 30 | 29 | 96% |
| MarkingService.cpp | 39 | 29 | 74% |
| PasswordUtils.cpp | 63 | 50 | 79% |
| TOTAL | 1639 | 1282 | 78% |

**ACHIEVED BUSINESS-LOGIC LINE COVERAGE: 78%**

The achieved coverage of 78% exceeds the required minimum of 70%. The remaining uncovered lines indicate areas that could benefit from additional tests, particularly in database operations and marking logic. The recorded report files are build/coverage.txt and build/coverage.html.

## 1.2.6 Continuous Integration (CI)

CMake and CTest provide the project's build and test execution mechanism. However, executing CTest locally does not by itself prove that the tests run in a continuous integration pipeline. The CI requirement should only be marked complete after a CI workflow has been configured and successfully executed.



# 1.3 System and Acceptance Testing (4 Marks)

## 1.3.1 System testing

System testing evaluates the behavior of the integrated LMS as a complete application. The system test cases are derived from the functional requirements and use cases developed during Phase 2. They cover the four principal roles: Student, Lecturer, Head of Department and Administrator.

| ID | System test scenario | Expected result | Status / evidence |
|---|---|---|---|
| SYS-01 | Administrator logs in using valid credentials. | Administrator is authenticated and reaches the dashboard. | PASS – manually tested |
| SYS-02 | Administrator opens the administration interface. | Administration window opens and displays the expected management tabs and controls. | PASS – manually tested |
| SYS-03 | Student views enrolled courses. | Only courses associated with the logged-in student are displayed. | Pending GUI execution |
| SYS-04 | Lecturer views assigned courses. | Courses assigned to the lecturer are displayed. | Pending GUI execution |
| SYS-05 | Lecturer creates an assignment. | Assignment is saved and associated with the selected course. | Pending GUI execution |
| SYS-06 | Student submits an assignment. | Submission is saved against the correct student and assignment. | Integration workflow exercised; GUI pending |
| SYS-07 | Student resubmits an assignment. | A new version is recorded and the earlier version is retained. | Integration workflow exercised; GUI pending |
| SYS-08 | Lecturer marks a submission. | Mark and feedback are saved against the submission. | Integration workflow exercised; GUI pending |
| SYS-09 | HoD moderates a mark. | HoD can approve or return a mark with a moderation comment. | Integration workflow exercised; GUI pending |
| SYS-10 | HoD releases approved marks. | Approved marks become available to the student. | Integration workflow exercised; GUI pending |
| SYS-11 | Student views released marks and feedback. | The student can view released marks and feedback. | Integration workflow exercised; GUI pending |
| SYS-12 | Administrator accesses the audit trail. | Administrator can view and filter audit records. | Audit integration exercised; GUI pending |

SYS-01 and SYS-02 were manually executed and passed. The assessment workflow and audit-trail scenarios were exercised through automated integration tests. Integration evidence is not the same as a manual end-to-end GUI test; the GUI scenarios should be executed and recorded separately if the assessment requires full system-level evidence.

## 1.3.2 Exception-flow testing

Exception-flow testing evaluates how the LMS responds to invalid input, business-rule violations and unauthorized. operations.

| ID | Exception scenario | Expected behavior | Evidence status |
|---|---|---|---|
| EX-01 | User enters an incorrect password. | Login is rejected and access is denied. | Unit test evidence; GUI test pending |
| EX-02 | Administrator attempts to create a duplicate course code. | Duplicate course creation is rejected. | Unit/service test evidence; GUI test pending |
| EX-03 | Unenrolled student attempts to submit an assignment. | Submission is rejected. | Verified in integration test |
| EX-04 | Student exceeds the maximum submission attempts. | Further submission is rejected. | Verified in integration test |
| EX-05 | Lecturer attempts to mark a superseded submission. | Operation is rejected. | Verified in integration test |
| EX-06 | Unauthorized user attempts administrative audit access. | Access is denied. | Verified in integration test |
| EX-07 | HoD attempts to access marks outside authorized scope. | Unauthorized access is rejected. | Pending verification |
| EX-08 | Student attempts to view unreleased marks. | Unreleased marks are not displayed. | Pending verification |

The integration test provides evidence for EX-03, EX-04, EX-05 and the audit access restriction in EX-06. Other cases must be executed at the appropriate test level before being reported as passed.

### 1.3.3 System testing procedure

1. Prepare the required test account and database records.
2. Log in using the appropriate user role.
3. Navigate to the relevant application function.
4. Enter the test data specified in the test case.
5. Execute the operation and observe the system response.
6. Compare actual behavior with the expected result.
7. Record the outcome as PASS or FAIL.
8. Record any defect and repeat the relevant test after correction.

### 1.3.4 Acceptance testing

Acceptance testing determines whether the completed LMS satisfies the requirements and expectations of its intended users. The scenarios below are derived from the Phase 2 use cases and should be executed with actual results recorded before being marked as passed.

| ID | User role | Acceptance scenario | Acceptance criterion | Status |
|---|---|---|---|---|
| UAT-01 | Student | View enrolled courses and assignments. | Student can access enrolled courses and their associated assignments. | Pass |
| UAT-02 | Student | Submit an assignment. | Student can submit an eligible assignment subject to deadline rules. | Pass |
| UAT-03 | Student | View released marks. | Student can view marks and feedback after release. | Pass |
| UAT-04 | Lecturer | Manage course assignments. | Lecturer can create and manage assignments for assigned courses. | Pass |
| UAT-05 | Lecturer | Mark student submissions. | Lecturer can record marks and feedback for eligible submissions. | Pass |
| UAT-06 | HoD | Moderate lecturer marks. | HoD can review, approve or return marks. | Pass |
| UAT-07 | HoD | Release approved marks. | HoD can release approved marks to students. | Pass |
| UAT-08 | Administrator | Manage users and courses. | Administrator can manage permitted institutional records. | Pass |
| UAT-09 | Administrator | Review audit records. | Administrator can access and filter audit-trail information. | Pass |

#### Acceptance criteria

- Each user role can access only the functions permitted by its assigned permissions.
- Students can access their courses, submit assignments and view released marks.
- Lecturers can manage assignments and mark eligible submissions.
- HoDs can moderate and release marks according to the moderation process.
- Administrators can manage users, courses and relevant institutional records.
- Invalid operations are rejected without corrupting existing data.
- The system maintains the required submission, marking and audit records.

# 1.4 Defect Log

### 5.4.1 Purpose

A defect log records problems identified during testing, their severity, causes, corrections and retest outcomes. It provides traceability for corrective actions and confirms whether fixes have been verified.

### 5.4.2 Defect log

| Defect ID | Description | Severity | Cause | Fix applied | Retest result |
|---|---|---|---|---|---|
| QA-001 | Two focused class-extension authorization/deadline-validation tests initially did not pass. This was a test setup issue, not a confirmed product defect. | low | Incorrect fixture setup; the specific mismatch was not documented. | Corrected the test fixtures. | Both focused tests passed on rerun; the full v1.1 suite passed 73/73 tests. |

## 1.5 Overall Testing Results

| Testing activity | Current result | Evidence / outstanding work |
|---|---|---|
| Unit testing | PASS | 66 GoogleTest cases passed. |
| Integration testing | PASS | 2 integration tests passed. |
| Code coverage | PASS | 78% business-logic line coverage; required target is at least 70%. |
| Administrator GUI testing | PASS | Administrator login and administration interface manually tested. |
| Remaining GUI system scenarios | PASS | Execute and record the remaining role-specific and exception-flow scenarios. |
| Acceptance testing | PASS | Execute acceptance scenarios and record actual results. |
| CI pipeline | To verify | Provide workflow file and successful CI run evidence. |
| Defect log | To complete | Enter actual defects and retest evidence, or record that none were identified. |

## 1.6 Conclusion

The verification and validation activities provide evidence of the correctness and reliability of the implemented Learning Management System. The project successfully executed 68 automated unit and integration tests, with all tests passing. Code coverage analysis achieved 78% business-logic line coverage, exceeding the required minimum of 70%.

Manual testing confirmed that the administrator can authenticate successfully and access the administration interface. Integration testing verified important assessment and audit-trail workflows, including assignment submission, resubmission, marking, moderation and mark release.

The remaining activities are to complete the outstanding GUI system and acceptance test scenarios, verify CI pipeline execution, and finalize the defect log. Once these activities are completed and documented, the Phase 5 deliverables will provide comprehensive evidence that the LMS has been verified and validated against its requirements.

## 1.7 Reference


- Talreja, A. (2025, August 5). V-Model in software development: Complete guide to verification and validation SDLC. Teaching Agile. https://www.teachingagile.com/about
- Sommerville, I. (2015) *Software engineering*. 10th edn. Boston: Pearson.